If you later attempt to load or query this vector store using a different embedding model that produces vectors of a different dimension, you will encounter problems.

Listing 8.6 Reproducing a vector dimension mismatch

```
const createVectorStore = async (documents) => {
  const splitter = new RecursiveCharacterTextSplitter({
    chunkSize: 1500,
    chunkOverlap: 200,
  });
  const documentChunks = await splitter.splitDocuments(documents);

  const embeddings =
new GoogleGenerativeAIEmbeddings({ apiKey,
model: ' gemini-embedding-exp-03-07' });
  return HNSWLib.fromDocuments(
    documentChunks,
    embeddings
  );
};

const loadVectorStoreWithDifferentEmbeddings = async () => {
  const differentEmbeddings =
new GoogleGenerativeAIEmbeddings(
{ apiKey, model: ' models/embedding-001' });
  const vectorStore = await HNSWLib.load(VECTOR_STORE_INDEX, differentEmbeddings);

  const results =
await vectorStore.similaritySearch('query', 5);
};
```

Creates embeddings using Google generative AI with a specific model for saving

Initializes a different embedding model for loading

The query here fails with "Error: Query vector must have the same length as the number of dimensions (0)."

To run this example, you must first navigate to the correct directory and then execute the script:

```
$ cd ch08/chat-langchain-troubleshooting
$ node embeddings-error-handling.js
```

The challenge here is that the two models may produce vectors of different dimensions, which results in compatibility problems during queries. For instance, the error message "Query vector must have the same length as the number of dimensions (0)" indicates that the dimensions of the query vector do not match what the vector store expects. To avoid this, you must ensure you use the same embeddings model in both retrieval and storage operations.

Missing integration problems

Often, certain features may not be supported by specific models or integrations, leading to runtime errors. For example, a common error you may find is when you want to generate structured outputs with JSON objects. Some providers do not support this feature, so the relevant calls will fail: